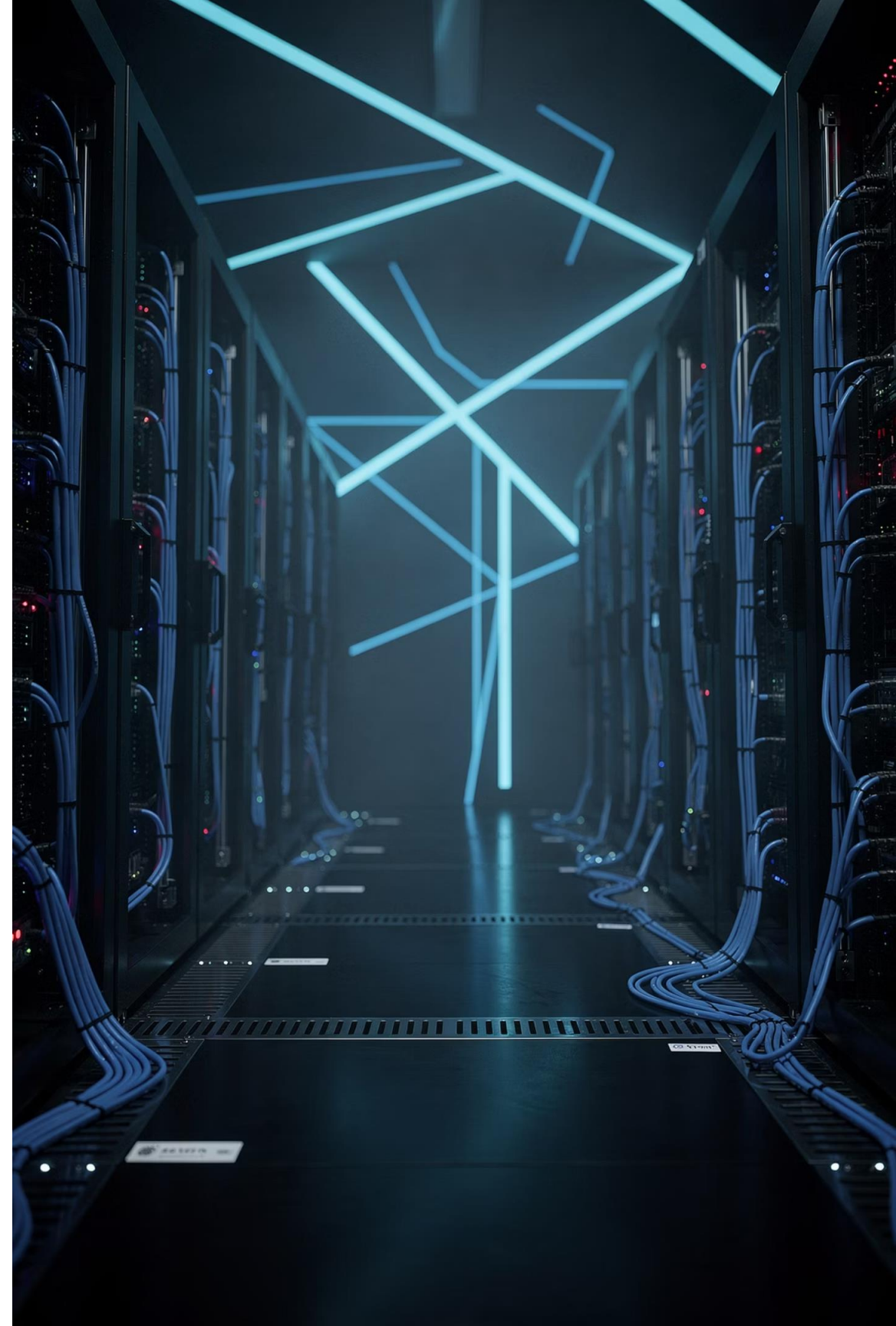


Angular Moderno

18 - Interceptors

Angular Moderno

18 - Interceptors:
Token, Logs e
Tratamento Global



Objetivo da Aula

Nesta aula vamos introduzir um dos conceitos mais importantes de aplicações reais: **Interceptors HTTP**. Veja o que vamos aprender:



Token Automático

Interceptar todas as requisições HTTP e adicionar token automaticamente



Logs Globais

Criar logs centralizados para todas as requisições e respostas



Tratamento de Erro

Implementar tratamento de erro centralizado em um único ponto



Contexto: até agora usamos `HttpClient` via Service. Agora vamos evoluir para **controle global** da aplicação.



Problema Atual

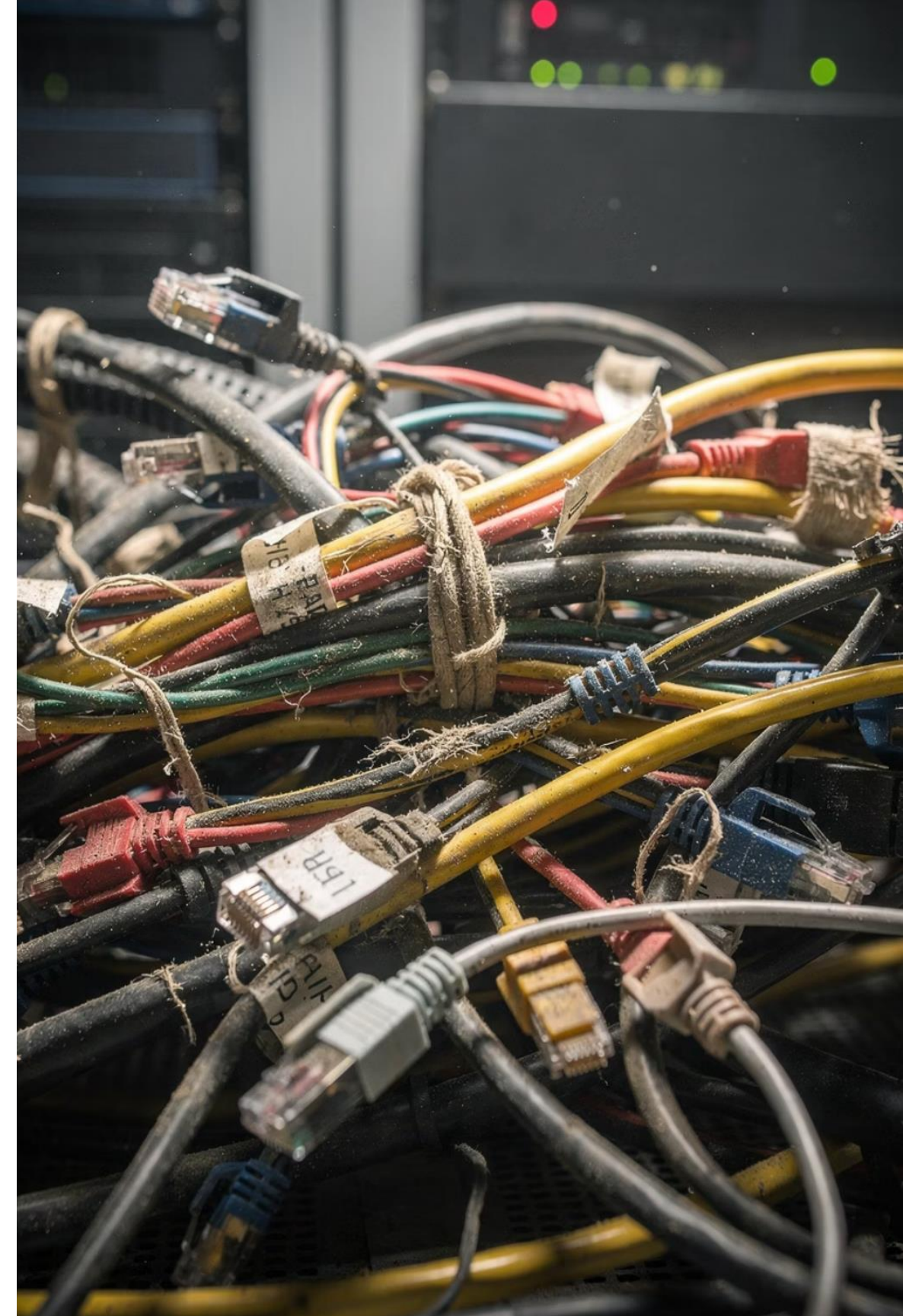
Situação atual do projeto: uma arquitetura que não escala:

Problemas Identificados

- Cada requisição HTTP está **isolada**
- **Não existe** controle global
- Token **não é aplicado** automaticamente
- Logs precisam ser feitos **manualmente**
- Tratamento de erro **repetido** em vários pontos

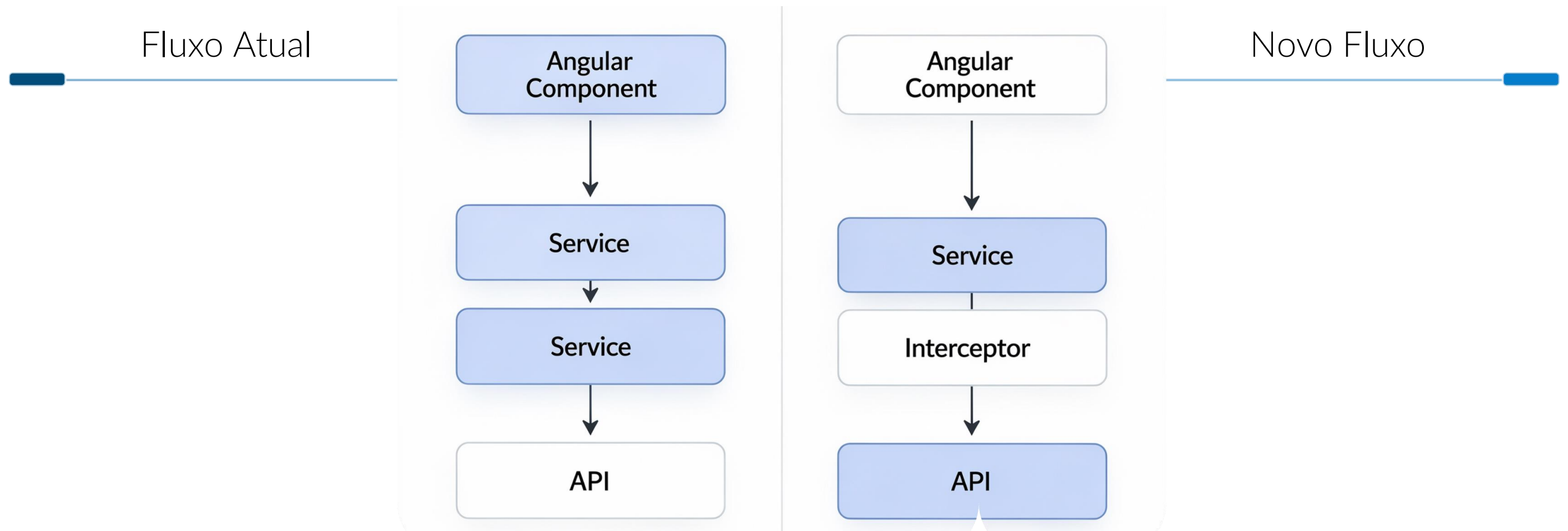
Problema Central: Arquitetura não escalável.

Cada serviço precisa reimplementar comportamentos que deveriam ser globais.



Conceito de Interceptor

O **Interceptor** é um middleware do Angular que intercepta todas as requisições HTTP antes de chegarem à API.



Com o Interceptor, centralizamos comportamentos globais como autenticação, logs e tratamento de erros em um único ponto da aplicação.

O interceptor atua entre o Service e a API, sem alterar o código do componente.

👍 **Benefício Principal:** Centralizar comportamentos globais — escreva uma vez, aplique em toda a aplicação.



Etapa 0: Parar o Servidor

Antes de iniciar alterações estruturais, é necessário parar o servidor de desenvolvimento:

`ng serve` → PARAR com **CTRL + C**

Evitar Erros de Build

Previne conflitos durante a criação de novos arquivos

Reconhecer Novos Arquivos

Garante que o compilador detecte os novos interceptors

Boa Prática Profissional

Padrão adotado em projetos reais de produção

Etapa 1: Criar o Interceptor

Atenção: Criar manualmente a pasta `interceptors` dentro de `core`, caso não exista.

Arquivo a Criar

```
src/app/core/interceptors/  
http.interceptor.ts
```

Justificativa

- Interceptor é global
- Pertence à camada `core`
- Separação de responsabilidades

Código Completo

```
import { HttpInterceptorFn } from '@angular/common/http';  
  
export const httpInterceptor: HttpInterceptorFn = (req, next) => {  
  console.log('Interceptando requisição:', req.url);  
  
  return next(req);  
};
```

 **HttpInterceptorFn** é o padrão moderno do Angular. Não usar classes (Angular antigo).

Etapa 2: Registrar o Interceptor

Arquivo: `src/app/app.config.ts`

Código Completo e Atualizado

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withFetch, withInterceptors } from '@angular/common/http';
import { provideClientHydration, withEventReplay } from '@angular/platform-browser';

import { routes } from './app.routes';
import { httpInterceptor } from './core/interceptors/http.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideClientHydration(withEventReplay()),
    provideHttpClient(withFetch(),
      withInterceptors([httpInterceptor])),
  ],
};
```

Como Funciona

- 1 `withInterceptors()`
Registra os interceptors na aplicação
- 2 Execução Global
Executa automaticamente para **todas** as requisições

Etapa 3: Testar o Interceptor

01 Rodar a Aplicação

```
ng serve
```

02 Acessar a Rota

Navegar para `/produtos` no navegador

03 Abrir o Console

Pressionar `F12` e acessar a aba Console

04 Verificar Resultado

Interceptando requisição: `https://fakestoreapi.com/products`

Se não aparecer log no console, o interceptor NÃO está registrado corretamente.

Objetivo: Validar que o interceptor está ativo e interceptando as requisições corretamente.



Etapa 4: Adicionar Token

Arquivo: `src/app/core/interceptors/http.interceptor.ts`

Código Completo Atualizado

```
import { HttpInterceptorFn } from '@angular/common/http';

export const httpInterceptor: HttpInterceptorFn = (req, next) => {
  // TOKEN
  const token = 'fake-jwt-token';

  const novaReq = req.clone({
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  return next(novaReq);
};
```

Conceitos-Chave

req é imutável

Não é possível modificar a requisição original diretamente

clone() é obrigatório

Cria uma cópia modificável da requisição

Token automático

Aplicado em **todas** as requisições sem esforço extra

Etapa 5: Adicionar Logs

CONTINUE EDITANDO o arquivo: `src/app/core/interceptors/http.interceptor.ts`

```
import { HttpInterceptorFn } from '@angular/common/http';
import { tap } from 'rxjs';

export const httpInterceptor: HttpInterceptorFn = (req, next) => {

  // TOKEN
  const token = 'fake-jwt-token';

  const novaReq = req.clone({
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  // SEGUIR COM A NOVA REQUEST + LOG RESPONSE
  return next(novaReq).pipe(
    tap({
      next: (event) => console.log('RESPONSE:', event),
      error: (error) => console.error('ERRO:', error)
    })
  );
};
```

tap() observa o fluxo
Não altera o comportamento da requisição

Logs centralizados
REQUEST e RESPONSE em um único lugar

Sem efeitos colaterais
Apenas observa, nunca interfere no fluxo

Etapa 6: Tratamento Global de Erro

Arquivo: `src/app/core/interceptors/http.interceptor.ts` — versão final completa:

```
import { HttpInterceptorFn } from '@angular/common/http';
import { tap, catchError, throwError } from 'rxjs';

export const httpInterceptor: HttpInterceptorFn = (req, next) => {
  // LOG REQUEST
  console.log('REQUEST', req.url);

  // TOKEN
  const token = 'fake-jwt-token';
  const novaReq = req.clone({
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  // SEGUE COM A NOVA REQUEST + LOG RESPONSE
  return next(novaReq).pipe(
    tap({
      next: (event) => console.log('RESPONSE:', event),
      error: (error) => console.error('ERRO:', error),
    }),
    catchError((error) => {
      console.error('ERRO GLOBAL:', error);

      if (error.status === 401) {
        console.warn('Não autorizado!');
      }
      if (error.status === 500) {
        console.warn('Erro interno do servidor!');
      }
      return throwError(() => error);
    })
  );
};
```

Por que catchError?

- Tratamento centralizado
Um único ponto para todos os erros HTTP
- Evita duplicação
Sem repetir lógica de erro em cada service
- Erros por status
401 = Não autorizado, 500 = Erro interno

Etapa 7: Simular Erro (Teste Controlado)

Arquivo: `src/app/features/produtos/produtos.service.ts`

Alteração no Service

```
// ETAPA 7 – alterar URL para simular erro  
private API = 'https://fakestoreapi.com/products-erro';
```

  Esta é uma alteração **proposital** para validar o comportamento do interceptor em caso de erro.

O que Esperamos Ver no Console

01 REQUEST

Log da URL com erro sendo interceptada

02 ERRO

Log do tap capturando o erro

03 ERRO GLOBAL

Log do catchError com status 404

Resultado do Teste de Erro

1	REQUEST URL com erro sendo interceptada e logada no console
2	ERRO Capturado pelo <code>tap</code> — <code>console.error('ERRO:', error)</code>
3	ERRO GLOBAL Capturado pelo <code>catchError</code> — <code>console.error('ERRO GLOBAL:', error)</code>

 Inspetor

 Console

 Debugger

 Rede

 Editor de estilos

 Desempenho

 Memória

 Armazenamento

 Acessibilidade

 Aplicação

 4







 Filtrar saída

Erros

Warnings

Info

Logs

Debug

CSS

XHR

Requisições



[vite] connecting...

[vite] connected.

Angular is running in development mode.

REQUEST <https://fakestoreapi.com/products-erro>

RESPONSE: ▶ `Object { type: 0 }`

Lista de produtos alterada: ▶ `Array []`

Valor total atualizado: `0`

▶ XHR GET <https://fakestoreapi.com/products-erro>

! ▶ ERRO:
▶ `Object { headers: {...}, status: 404, statusText: "Unknown Error", url: "https://fakestoreapi.com/products-erro", ok: false, type: undefined, redirected: false, responseType: "cors", name: "HttpErrorResponse", message: "Http failure response for https://fakestoreapi.com/products-erro: 404 ", ... }`

! ▶ ERRO GLOBAL:
▶ `Object { headers: {...}, status: 404, statusText: "Unknown Error", url: "https://fakestoreapi.com/products-erro", ok: false, type: undefined, redirected: false, responseType: "cors", name: "HttpErrorResponse", message: "Http failure response for https://fakestoreapi.com/products-erro: 404 ", ... }`

! ▶ Erro ao carregar produtos:
▶ `Object { headers: {...}, status: 404, statusText: "Unknown Error", url: "https://fakestoreapi.com/products-erro", ok: false, type: undefined, redirected: false, responseType: "cors", name: "HttpErrorResponse", message: "Http failure response for https://fakestoreapi.com/products-erro: 404 ", ... }`

Angular hydrated 2 component(s) and 61 node(s), 0 component(s) were skipped. Learn more at <https://v21.angular.dev/guide/hydration>.

client:733:9

client:827:12

_debug_node-chunk.mjs:10992:13

http.interceptor.ts:6:11

http.interceptor.ts:20:32

lista-produtos.ts:61:15

lista-produtos.ts:65:15

[HTTP/2 404 274ms]

http.interceptor.ts:21:33

http.interceptor.ts:24:15

lista-produtos.ts:120:17

_debug_node-chunk.mjs:10992:13

>>



Etapa 8: Restaurar a API

Arquivo: `src/app/features/produtos/produtos.service.ts`

Código Restaurado (IMPORTANTE RESTAURAR)

```
private API = 'https://fakestoreapi.com/products';
```

- ✔ Restaurar a URL correta para retomar o funcionamento normal da aplicação.

Por que Restaurar?

- O teste de erro foi concluído com sucesso
- O interceptor foi validado
- A aplicação deve voltar ao estado funcional
- Boa prática: nunca deixar código de teste em produção

Resultado Após Correção

REQUEST

URL correta interceptada e logada
com sucesso

RESPONSE

Resposta HTTP 200 capturada pelo
tap

Dados Carregados

Lista de produtos exibida corretamente na tela

 Inspetor

 Console

 Debugger

 Rede

 Editor de estilos

 Desempenho

 Memória

 Armazenamento

 Acessibilidade

 Aplicação







 Filtrar saída

Erros

Warnings

Info

Logs

Debug

CSS

XHR

Requisições



[vite] connecting...

[vite] connected.

Angular is running in development mode.

REQUEST <https://fakestoreapi.com/products>

RESPONSE: ▶ [Object { type: 0 }](#)

Lista de produtos alterada: ▶ [Array \[\]](#)

Valor total atualizado: 0

RESPONSE: ▶ [Object { headers: {...}, status: 200, statusText: "OK", url: "https://fakestoreapi.com/products", ok: true, type: 4, redirected: false, responseType: "cors", body: \(20\) \[...\] }](#)

Lista de produtos alterada: ▶ [Array\(20\) \[{...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, ... \]](#)

Valor total atualizado: 3240.91999999999987

Angular hydrated 22 component(s) and 241 node(s), 0 component(s) were skipped. Learn more at <https://v21.angular.dev/guide/hydration>.

>>



Conclusão da Aula

Nesta aula construímos um interceptor completo e funcional. Veja o que foi aprendido:



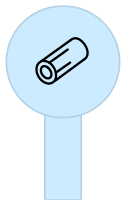
Interceptor Funcional Moderno

Usando `HttpInterceptorFn` — padrão atual do Angular



Token Automático

Adicionado via `req.clone()` em todas as requisições



Logs Globais

REQUEST e RESPONSE centralizados com `tap()`



Tratamento de Erro Centralizado

Erros 401 e 500 tratados com `catchError()`



Arquitetura Final: Component → Service → **Interceptor** → API



Nenhuma alteração foi feita no componente. Toda a lógica é centralizada.

Dúvidas

?

